

Design

The design phase is the process of designing exactly how the specifications are to be implemented. It defines specifically how the software is to be written including an object model with properties and methods for each object and a detailed database design.

- ✓ Create State Diagrams and Documents
- ✓ Create Activity Diagrams and Documents
- ✓ Create Sequence Diagrams and Documents
- ✓ Create Collaboration Diagrams and Documents
- ✓ Performance Testing
- ✓ Demonstrate Technical Feasibility

- Ensure that standards have been established for depicting designs (such as data flow diagram models for process-oriented systems, or entity-relationship models for data-oriented systems), and that the standards are being followed.
- Ensure that changes made to the designs are properly controlled and documented.
- Ensure that coding doesn't begin until the system design components have been approved according to agreed-upon criteria. Of course, we all do some coding before we really "should"; as long as you think of it as "prototyping", that's fine. Just don't get carried away prematurely.
- Ensure that design reviews proceed as scheduled.

Design Phase

This phase specifies exactly the components needed to deliver all items in the functional specification. Our design methodology includes a 3-tier design:

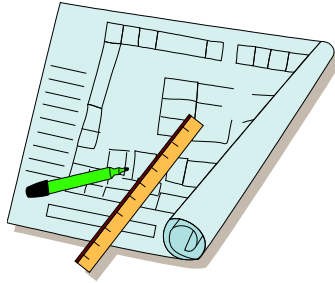
1. **Presentation Layer** – Visual Basic Forms or Web Pages. No business rules appear in this layer.
2. **Business Layer** – Visual Basic Active Servers (DLL or EXE) that encapsulates all the business rules.
3. **Database Layer** – Can be any ODBC or OLE DB compliant database.

The design phase specifies all components needed to deliver the system in a 3-tier fashion. Upon completion of this phase, a project plan may be developed that details the costs, effort and timeframes in which the solution may be delivered. As a final step, the client may decide to add, omit or delay functionality to fit within their timeframe and budget.

Deliverables

- Architecture Document
- Detailed Design Document
- Test Design
- Object Model
- Data Model
- Project Plan
- Budget
- Customer, Development, Testing Signoff

[ProductName] Architectural Overview



Revision History

Date	Reason for change(s)	Author(s)
09/15/1988	First Draft	John Doe

Table of Contents

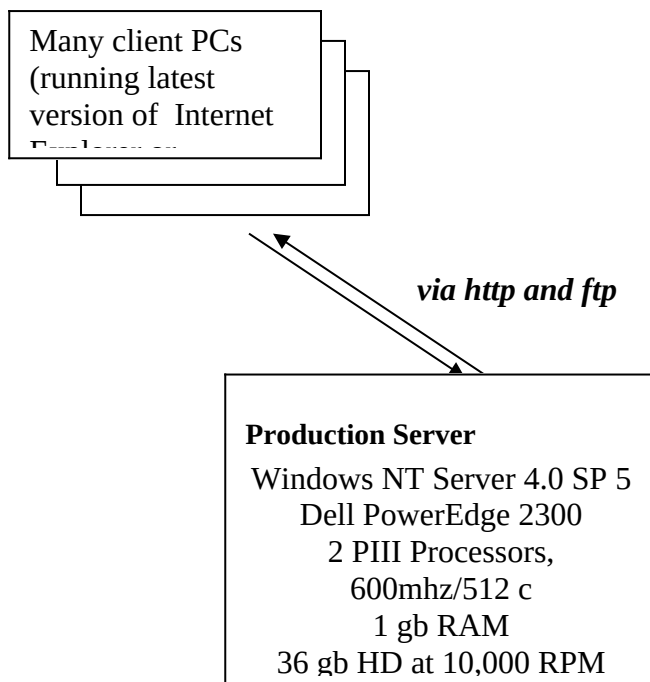
Revision History.....	1
Overview.....	2
Hardware Design.....	2
Security Design.....	2
Application Perspective.....	2
Credit Card Perspective.....	3
Server Perspective.....	3
Software Component Design.....	3
User Services.....	3
Web Interface.....	3
User Friendly Interface.....	4
User Configurable Interface.....	4
Easy Maintenance.....	4
Good Performance.....	4
Object Model.....	5
User Services.....	5
Business Services.....	6
psBusiness.DLL.....	6
Data Services.....	7
Retrieve List.....	7
Update a Record.....	7
Add A Record.....	7
Delete a Record.....	8
App Builder.....	9

Overview

This document describes the architectural roadmap for implementing software solutions for [CompanyName]. It specifically addresses the design for the [ProductName] product, but is used as a framework for creating new products.

Hardware Design

Although this is designed as a 3-tier solution, the hardware contains many client workstations connected to one server that hosts both IIS and SQL Server. If bottlenecks appear in the future, multiple servers can be added to allow IIS, MTS and/or SQL Server to have their own machines. If needed, we could also create a server farm of IIS and MTS machines. The current hardware configuration is as follows:



Security Design

Security has been built into the design from an application, credit card protection and a server perspective.

Application Perspective

The application allows a security administrator to be setup that is responsible for administering user accounts. The security administrator can also set up other security administrators as well. When defining security, the security administrator can give 4 levels of security for any entity within the system (an entity might be contacts, projects, defects, functional specifications, etc.):

- Add – Gives the user the ability to add records to the entity.
- Update – Gives the user the ability to update records in the entity.
- Delete – Gives the user the ability to delete records from the entity.
- Read – Gives the user the ability to access (read) data from the entity.

This type of security is performed via a programming model that allows security to be established and maintained. The programming model is flexible and well encapsulated and allows the security administrator to revoke rights and change passwords.

Credit Card Perspective

When credit cards must be processed, they are sent to a third party company via secure socket layers. Secure socket layers encrypt the credit card information prior to leaving our server and decrypts when it reaches the third party. Likewise, when the authorization is sent back, it is also done via secure socket layers.

Server Perspective

From the Windows NT and SQL Server perspective, all commonly used accounts have been locked down. For example, most often business's have a userid named *Administrator* for Windows NT and *SA* for SQL Server. Many companies do not take the necessary steps to secure these accounts creating a security breach for hackers. We have secured those accounts and have secured the file system as well. In addition, we do not set each user up with a userid in SQL Server. Instead, we use impersonation. Impersonation allows the user to enter their email address and password to access the application, but it does not actually use any of that information to connect to the SQL Server database. Instead, there is a single account established that is used for all database access within the application. This has several advantages. It eases SQL Server administration, prevents the users from logging into the database via an ODBC or OLE-DB provider, allows connection pooling, and prevents destructive rights from the user population (for example, users can not create or drop tables or system objects).

Software Component Design

The software architectural design used by [CompanyName] is a 3-tier design:

1. User Services – This layer is responsible for displaying information to the end user.
2. Business Services – This layer is responsible for enforcing business rules.
3. Data Services – This layer is responsible for retrieving and storing database items.

User Services

This layer is responsible for displaying information to the end user. The goals for the user interface are as follows:

1. Web Interface
2. User Friendly
3. User Configurable Interface
4. Easy Maintenance
5. Good Performance

Web Interface

The interface is accessible from the user's browser, both Netscape and Microsoft Explorer clients. To access [ProductName], the user navigates to [ProductWebPage].

To accomplish this goal, [CompanyName] has an HTML home page that contains information about the company and links to each product sold. Each product page has information regarding the product, helpful tips for using the product and the information covering the subject area of the product. For example, the [ProductName] product web page contains documentation on testing methodology and sample templates to be used for creating master test plans, detailed test plans, test cases and project plans.

User Friendly Interface

The interface must be easy to use and provide a good experience for the end user. This includes the ability to specify user preferences, screens that have a clean look and feel, screens that are resizable as to allow the user to display the application in the resolution they desire, and consistency between all areas of the application.

To accomplish this goal, the development is done using Visual Basic web classes, Active Server Pages and HTML. This allows the user to use most any browser and operating system. Although most any browser can be used, the system has been extensively testing with the latest versions of Netscape and Internet Explorer.

User Configurable Interface

The Interface must allow the user to add custom fields to areas of the application as to make the software extensible. For example, if the user would like to track information that is not currently stored in the software, they should be able to define new fields and set parameters for tracking the information. Also, the users should be able view a history of changes made to the major areas of the system and view errors encountered

over time. The user should be able to sort and filter any grid and store the sort and filter parameters for later use.

To accomplish this goal, tables are created to store custom fields, audits, filters, sorts, and preferences. Access to the tables is wrapped by a Business Services DLL as to allow the programmer ease of use for these functions.

Easy Maintenance

The solution set should be easily maintainable once in production as to allow minimal effort for supporting the applications.

To accomplish this goal, most of the application is written in Visual Basic using classes to encapsulate logic. In addition, many support functions are added to ease the production support tasks. This includes a Frequently Asked Questions area that allows the user to search a knowledge base repository of questions to get a quick answer. If the answer cannot be found, the user sends a question to the support staff that will answer the question and add it to the FAQs area. Additionally, a suggestion box is available to allow the user to request new functionality. Once a suggestion is added, other users can vote on the suggestion. Once enough support is generated for the new functionality, it will be added to the system.

No phone number will be published for production support. Having all support handled through FAQs and email will significantly decrease the support effort needed.

Good Performance

Response time must be good as to not make the user wait too long for screens to display. For tasks that are long by nature, a progress bar should appear to notify the user as to the progress of the task. Also, a history of performance by user and project should be easily accessible.

To accomplish this goal, the logic will be compiled into 4 DLLs:

1. psCommonASP.dll – This DLL contains the Visual Basic web class that is used to present the user interface via web pages. This DLL resides on the server.
2. psCommonUtils.dll – This DLL is used for common logic. This provides all tiers quick response to common logic. It is called by all other DLLs. This DLL can run on the client or server, but in our case runs on the server.
3. psBusiness.dll – This DLL contains all the business rule logic and access to the database. This DLL runs on the server under MTS.
4. psDatabase.dll – This DLL contains a thin database layer that handles running SQL and stored procedures. It also performs updates to the database and handles concurrency checking. This DLL resides on the server.

To monitor performance, performance reports are available that show the average response time for each task. The client can drill down further and see performance by user as to see which users are experiencing bottlenecks. Other performance reports are available as well

Object Model

User Services

psCommonASP.dll

This DLL is responsible for rendering HTML based on the page requested. This DLL is a Visual Basic web class that allows us to easily create HTML documents with static and dynamic content. Since this is a web class, it runs on the Server.

nsCommonASP

psCommonUtils.dll

This DLL has 3 common classes used by all the other DLLs to simplify logic. These could have been put in Visual Basic modules, but were put in classes to make distribution and re-use easier. Common.cls contains common logic used by all applications. MSEXcel.cls encapsulates the Microsoft Excel object model to make it easier to write spreadsheets. MSWord.cls encapsulates the Microsoft Word object model to make it easier to write to Word documents. There is no hierarchy between these objects, as they are independent entities. This DLL is designed to run both on the client and the server. In our case, it runs on the Server.

Common

MSEXcel

MSWord

Business Services

psBusiness.DLL

This layer is responsible for packaging information and enforcing business rules needed by User Services. The business layer has a hierarchy that allows for easy access to

common functions. There are many classes in this layer, so they will not be listed here. There is no hierarchy between these objects, as they are stateless independent entities.

Retrieval of read-only data is performed by disconnected recordsets. These objects are defined in plural form (i.e. Companies, Depts, etc.). They are client-side forward-only Active Data Object (ADO) recordsets and are used to display lists of data on the screen. These objects have a set of common methods used for adding/editing data:

1. Add – This method adds a record to the database. All Adds are done with Stored Procedures named sp_Add_XXXXX where XXXXX is the table name.
2. GetForUpdate – This method returns a row of data in an updateable, dynamic disconnected recordset. Once the data is updated in the recordset, the Update method is invoked on the object (not the recordset itself). All GetForUpdates are done with Stored Procedures named sp_GetForUpdate_XXXXX where XXXXX is the table name
3. Update – This method updates the data in the recordset, checking for concurrency issues. If the record is stale, an error is raised allowing the client to see what things have been updated since the last time they attempted to update the record. All data is validated via the Validate method prior to performing the update. No stored procedure is used for this since we are updating the recordset directly.
4. Delete – This method deletes a record based on a primary key. All data is validated via the Validate method prior to performing the delete. All Deletes are done with Stored Procedures named sp_Delete_XXXXX where XXXXX is the table name.
5. Validate – Enforces all business rules and integrity constraints. If violations are found, an error is raised.
6. CheckSecurity – This method is used to check the security level of the user trying to perform add/update/deletes. If security is violated, the user is not allowed to persist data.

This DLL is designed to run on the server under Microsoft Transaction Server (MTS).

Data Services

The Data Services layer is responsible for retrieving, adding, updating and deleting information from the database. The CommonDb.cls (part of psDatabase.dll) encapsulates the data access methods used by Data Services.

Retrieve List

Sets of data are retrieved using the CommonDb.RunSQLReturnRS method. This was chosen rather than stored procedures because the system needs to be flexible enough to create queries with user-defined filters and sorts. There is no way to allow the where clause and order by clause to be dynamically set in a stored procedure, therefore, this method is employed. The recordset returned by this method is used to populate lists and combo boxes in the user interface.

Update a Record

If the client wants to update data, they select a record from a list in the user interface and User Services calls the Business Services GetForUpdate method on the associated object. This method invokes the CommonDb.RunSPReturnRS_RW method that returns an updateable recordset. Each table has a stored procedure used by these methods. The naming convention for the stored procedure is sp_GetForUpdate_xxxxxx where xxxxxx is the entity. For example, sp_GetForUpdate_Companies retrieves a single Company record to be updated. It also returns the SQL issued during the retrieval. This is then used by Business Services when the Update method is invoked on the object. That way it knows exactly what SQL was run during the GetForUpdate method. This allows it to determine if there are any concurrency crashes prior to issuing the update on the record. If any errors occur (including concurrency and integrity constraint issues), an error is raised. Below is an example of how a record is updated:

```
'Declarations
Dim rs as recordset
Dim objCompanies as psBusiness.Companies

'Return an updateable recordset
Set rs = new Recordset
Set objCompanies = new psBusiness.Companies
Set rs = objCompanies.GetForUpdate(Session,
CompanyId:=1)

'Update the company name in the recordset
rs("CompanyName") = 'XYZ Corp.'
```

```
'Apply the update by sending it the recordset you
updated
'Note: If concurrency clash happens, it will raise an
error
objCompanies.Update Session, rs
```

Add A Record

If the client wants to add data, they call the Business Services ADD method on the object. This method invokes the CommonDb.RunSPReturnKey method. No recordset is instantiated; the stored procedure simply runs. This makes adding records very responsive. Each table has a stored procedure named sp_Add_XXXXX where XXXXX is the table name. For example, sp_Add_Companies adds a record to the Companies table. It adds the table and returns the IDENTITY that was created as the primary key. If the table does not have an IDENTITY column, CommonDb.RunSP is invoked to process the add. If any errors occur (including concurrency and integrity constraint issues), an error is raised.

Delete a Record

If the client wants to delete a record, they call the Business Services DELETE method on the object. This method invokes the CommonDb. No recordset is instantiated; the stored procedure simply runs. This makes deleting records very responsive. Each table has a stored procedure named sp_Delete_XXXXX where XXXXX is the table name. For example, sp_Delete_Companies adds a record to the Companies table. No recordset is returned. If any errors occur (including concurrency and integrity constraint issues), an error is raised.

CommonDb.cls

One object that deserves some attention is the CommonDb.cls. This class is used to encapsulate database processing. It provides common methods for accessing ADO by running SQL and stored procedures. It also contains the connection parameters for SQL Server. Below are the common methods CommonDb.cls exposes:

1. RunSPReturnRS – Executes a stored procedure, handles the parameter passing and returns the results as a disconnected read-only recordset. This is used to populate lists of data on the screen.
2. RunSPReturnRS_RW - Executes a stored procedure, handles the parameter passing and returns the results as an updateable, batch optimistic, dynamic recordset. This is used to retrieve a single record for update.
3. RunSPReturnKey - Executes a stored procedure, handles the parameter passing and returns the SQL Server Identity used when the new record was created. This is used to call the stored procedures for adding records to the database.
4. RunSP – Runs a stored procedure and does not return anything (no recordset).

5. RunSQL – Runs any SQL and does not return anything (no recordset).
6. RunSQLReturnRS – Runs any SQL and returns the results as a disconnected read-only recordset. This is used to populate lists of data on the screen when filters and sorts are needed. This gives the added flexibility of defining the SQL just as you want it.
7. CollectParameters – This is a helper function that takes the tedium out of parameter passing in ADO.
8. GetConnectionString – This is the connection string for our SQL Server database.

App Builder

App Builder will build a set of classes and stored procedures for any or all the tables in the database.

The other functionality of AppBuilder is to create Visual Basic routines that create HTML. To use this, you simply go into MS Front Page and design the page as you need it. Then copy / paste the HTML into AppBuilder. It will create a VB routine to display the data.

Detailed Design



Owners and List of Contacts

Name	Email	Phone	Role
John Doe	Jdoe@Me.com	303-894-7315 W 303-471-8344 H 303-203-5567 Pgr	Project Manager Development Lead
Joe Tester			System Test Lead
Jane ProdSupport			Production Support Mgr
Joe UserMgr			User Test Lead
Joe Developer			Developer – Presentation Tier
Jane Developer			Developer – Business Tier
Joe DBA			Data Base Administrator
Joe Customer			Department VP
Jane Customer			Department Mgr
Josey Customer			Product Support

Signoffs

Phase	Name	Date	Signature
Detail Design	John Doe, PM/DM	xx/xx/xx	
	Joe Tester, System Test Lead		
	Jane ProdSupport, Production Support Mgr		
	Joe User Mgr, UM		
	Joe Customer, Customer		

Revision History

Date	Reason for change(s)	Author(s)
------	----------------------	-----------

09/15/1988	First Draft	John Doe
------------	-------------	----------

Table of Contents

Detailed Design.....	1
Owners and List of Contacts.....	1
Signoffs.....	1
Revision History.....	1
1. Summary.....	3
2. Hardware Requirements.....	3
3. Software Requirements.....	3
4. Presentation Layer.....	3
4.1 Screens.....	3
4.2 Reports.....	3
5. Business Layer.....	4
6. Database Layer.....	4
7. Other Design Considerations.....	4
7.1 Conversion Modules.....	4
7.2 Archive and Purge Modules.....	4
7.3 Backup and Recovery Design.....	4
7.4 Security Architecture.....	4
7.5 System Interfaces.....	4
7.6 Batch Jobs.....	4
7.7 Performance and Response Time Considerations.....	4
7.8 Platform Dependence and Installation Considerations.....	4
7.9 Localization Considerations.....	5
7.10 Other Modules.....	5
8. Detailed Design to Functional Requirement Cross Reference Matrix.....	5

1. Summary

This document describes the detailed architectural design for the system. It specifies all known components needed to deliver a complete system.

2. Hardware Requirements

List all hardware requirements needed to run the system. Include server and client configurations.

3. Software Requirements

List all software used to develop the system (Visual Basic, Visual Interdev, SQL Server, etc) along with any software necessary for the system to function (MS Excel, True DB Grid, Internet Explorer, etc).

4. Presentation Layer

This section describes all the screens and reports needed to deliver all the functional requirements. Include screen descriptions, screen shots, report descriptions and report shots. The client should understand that the final product may not be exactly as listed here but the functionality will stay the same. During coding, we may merge or separate screens to achieve a nicer user interface and to promote reusability of components.

4.1 Screens

4.1.1 Screen XYZ

Include screen shot here

Description	Describe the purpose of the screen as to ensure the client and developer agrees on its functionality.
Security Group	List the security groups that may use the screen
Data	Describe each field on the screen and how it is derived
Actions	Describe each action that can be performed from this screen. This usually involves command buttons, menu items, etc.

4.2 Reports

4.2.1 Report XYZ

Description	Describe the purpose of the report as to ensure the client and developer agrees on its functionality.
-------------	---

Security Group	List the security groups that may use the report
Data	Describe each field on the report and how it is derived
Groupings	Describe how the report should be grouped and/or sorted
Totals	Describe the sub and grand totals needed on the report
Filters	Describe any filter or selection criteria needed for the report. How is this selection criteria entered?
Export Formats	Describe the export format of the report (print preview, printer, MS Word, MS Excel, HTML, etc.)
Frequency	If this report should be run periodically, specify the frequency (Daily, Weekly, Monthly, Quarterly, Yearly, On Demand)

5. Business Layer

Define all of the objects necessary to support the presentation layer. First show the object hierarchy then include the Microsoft Visual Modeler file that shows the details as Appendix A.

6. Database Layer

Define all of the tables/columns/views/stored procedures necessary to support the business layer along with an Entity-To-Relationship diagram. Include the ERD file that shows the details as Appendix B.

7. Other Design Considerations

7.1 Conversion Modules

Describe any programs that are used to import, convert or munge data.

7.2 Archive and Purge Modules

Describe any programs that are used to perform special functions like purging and archiving.

7.3 Backup and Recovery Design

Describe the process for backing up and recovering lost data.

7.4 Security Architecture

Describe how security will be enforced and at what level (or groups). If using NT, address the NT security model, IIS security, Secure Socket Layers, etc. if applicable.

7.5 System Interfaces

Describe how we will feed or receive feeds from other systems. This may be via replication, BCP, SQL Loader, etc. Describe the data that will flow between the systems, how often it will be updated, and who the system owners for each system are.

7.6 Batch Jobs

Describe any jobs that will run regularly to perform batch based commands.

7.7 Performance and Response Time Considerations

Explain how we will design for maximum response time. This includes the use of Stored Procedures, de-normalized data (if applicable), server configuration (size, memory, etc), programming techniques, and the use of database tools such as Oracle's Explain Plan and SQL Server's Show Plan.

7.8 Platform Dependence and Installation Considerations

Explain the installation (or setup) process in which the client must use to get the system up and running along with our plan for ensuring it will run on all the platforms requested by the client (Windows 95, 98, NT 4, 5, etc).

7.9 Localization Considerations

Explain our design for handling issues specific to localization (European date and postal code format, etc.), if any.

7.10 Other Modules

Describe any other miscellaneous programs.

8. Detailed Design to Functional Requirement Cross Reference Matrix

To ensure all functional requirements have been captured in the detailed design, cross-reference each functional requirement with it's associated Detailed Design element.

Functional Requirement	Detail Design
4.1 Feature 1	3.1.1 Screen XYZ 3.2.1 Report XYZ